

Relatório Técnico: B3Tracker ML

1. Problema escolhido

O B3Tracker é uma aplicação web em Python/Django que monitora cotações de ativos financeiros da B3 e envia notificações por e-mail quando o preço ultrapassa limites superiores ou inferiores definidos manualmente. Esse mecanismo funciona como um túnel fixo de preço, mas apresenta uma limitação importante: os limites estáticos não acompanham mudanças de volatilidade, tendência ou comportamento recente do mercado.

O problema de engenharia escolhido foi transformar esse mecanismo de alerta em uma camada mais inteligente de apoio à decisão. Em vez de apenas verificar se o preço atual cruzou um limite manual, o sistema passa a classificar a janela atual do ativo como COMPRA, VENDA ou NEUTRO, com base em indicadores técnicos calculados a partir da série histórica.

2. Motivação

Em ativos financeiros, uma mesma variação de preço pode ter significados diferentes dependendo do contexto. Uma queda de 2% pode ser um movimento extremo em um ativo estável, mas pode ser normal em um ativo volátil. Da mesma forma, um preço acima da média móvel pode indicar tendência, mas também pode indicar exaustão se vier acompanhado de RSI elevado e volume anormal.

A motivação do projeto é melhorar a qualidade dos alertas do B3Tracker usando aprendizado supervisionado para combinar múltiplas evidências: retorno recente, tendência, volatilidade, volume e contexto de mercado. A proposta não é prever preços com precisão nem automatizar operações, mas reduzir a dependência de limites manuais e oferecer um alerta mais adaptativo.

3. Metodologia

3.1 Formulação do problema

O problema foi modelado como classificação multiclasse. Cada observação representa um ativo em uma data. As features são calculadas apenas com informações disponíveis até aquela data. O rótulo é criado olhando o retorno futuro em cinco pregões.

A regra de rotulagem foi:

```
COMPRA: retorno futuro de 5 dias >= limite dinamico
VENDA:  retorno futuro de 5 dias <= -limite dinamico
NEUTRO: caso contrário
```

O limite dinâmico é calculado como:

```
limite dinamico = max(2%, volatilidade_20d * sqrt(5))
```

Essa escolha evita que o mesmo limite absoluto seja aplicado a ativos com riscos muito diferentes. Também reduz o número de sinais gerados por variações pequenas que podem ser apenas ruído.

3.2 Engenharia de features

Foram criadas features técnicas clássicas de mercado, organizadas em grupos:

Grupo	Features principais	Justificativa
Retorno	retornos de 1, 3, 5, 10 e 20 dias	Capturam momentum ou reversão recente
Tendência	médias móveis e distância para médias	Medem posição do preço em relação à tendência

Grupo	Features principais	Justificativa
Momentum	RSI e MACD	Identificam sobrecompra, sobrevenda e força direcional
Volatilidade	volatilidade de 5, 10, 20 e 60 dias	Ajustam a leitura do movimento ao regime de risco
Bollinger	posição e largura das bandas	Combinam tendência e volatilidade
Volume	volume relativo e z-score de volume	Indicam se o movimento teve participação relevante
Mercado	retorno do BOVA11, volatilidade do índice e beta	Diferenciam movimentos isolados de movimentos sistêmicos

O cuidado principal foi evitar vazamento de dados. Indicadores como médias móveis, RSI e volatilidade usam apenas dados passados. O retorno futuro é usado apenas para construir o rótulo.

3.3 Modelos

O modelo principal recomendado é Random Forest, por ser robusto para dados tabulares, exigir pouca normalização e permitir análise de importância das variáveis. O projeto também permite executar XGBoost, que costuma ter bom desempenho em dados tabulares, mas foi mantido como alternativa para não aumentar a complexidade sem necessidade.

A escolha por modelos clássicos foi proposital. Redes neurais recorrentes ou LSTMs poderiam ser usadas em séries temporais, mas seriam menos transparentes, mais difíceis de calibrar e menos justificáveis para o tamanho do problema.

3.4 Separação temporal

A separação entre treino, validação e teste respeita a ordem cronológica dos dados. Isso é essencial em séries temporais financeiras, pois um split aleatório permitiria que o modelo aprendesse padrões do futuro para prever o passado.

O pipeline usa:

Partição	Uso
Treino	Ajuste do modelo
Validação	Escolha de limiares de decisão
Teste	Avaliação final

3.5 Tratamento do desbalanceamento

O conjunto tende a ser desbalanceado, com predominância da classe NEUTRO. Isso acontece porque a maior parte dos dias não apresenta movimento forte o suficiente para justificar uma compra ou venda. Para lidar com isso, foram adotadas três decisões:

1. uso de limite dinâmico baseado em volatilidade;
2. uso de modelos com ponderação de classes no Random Forest;
3. avaliação por matriz de confusão, precision, recall, F1 e acurácia balanceada, não apenas acurácia global.

Não foi adotado oversampling sintético como estratégia principal, pois em séries temporais financeiras exemplos artificiais podem criar padrões que não existiram no mercado.

4. Resultados esperados e interpretação

A execução do pipeline gera métricas em data/processed/, incluindo matriz de confusão e comparação entre previsão direta do modelo e previsão após aplicação de limiar conservador.

O ponto central da avaliação é a análise de risco dos erros:

Tipo de erro	Interpretação	Gravidade
NEUTRO previsto como COMPRA	Alerta de compra sem movimento futuro suficiente	Alta
VENDA previsto como COMPRA	Erro de direção em cenário negativo	Muito alta
COMPRA previsto como NEUTRO	Oportunidade perdida	Média
NEUTRO previsto como VENDA	Alerta de venda desnecessário	Alta

No contexto do B3Tracker, falsos positivos são mais graves do que falsos negativos. Um falso positivo pode induzir o usuário a tomar uma decisão ruim, enquanto um falso negativo apenas deixa de emitir um alerta. Por esse motivo, o projeto aplica um limiar mínimo de confiança:

$P(\text{COMPRA}) \geq 0.60$ e margem contra VENDA ≥ 0.15
 $P(\text{VENDA}) \geq 0.60$ e margem contra COMPRA ≥ 0.15

Essa calibragem tende a reduzir a quantidade de alertas, mas aumenta a seletividade. Do ponto de vista de engenharia, essa é uma troca aceitável para um sistema de notificação financeira, no qual excesso de alertas errados reduz a confiança do usuário.

5. Integração com o B3Tracker

A integração proposta não substitui imediatamente o alerta manual. O desenho mais seguro é híbrido:

Camada	Função
Túnel manual	Continua permitindo limites definidos pelo usuário
Modelo ML	Sugere sinal técnico de compra, venda ou neutralidade
Limiar de risco	Bloqueia alertas com baixa confiança
E-mail	Mantém o canal de notificação já existente

Com isso, o B3Tracker passa a enviar alertas mais informativos, por exemplo:

PETR4.SA: sinal COMPRA
Probabilidades: NEUTRO 22%, COMPRA 66%, VENDA 12%
Justificativa: tendência curta positiva, RSI moderado e volume acima da média.

6. Limitações

O modelo possui limitações relevantes. Ele usa apenas histórico de preços e volume, portanto não captura notícias, fatos relevantes, divulgação de resultados, mudanças regulatórias, decisões de política monetária ou eventos externos. Esses fatores podem dominar completamente os indicadores técnicos.

Outra limitação é que o comportamento do mercado muda ao longo do tempo. Um padrão aprendido em determinado período pode deixar de funcionar em outro regime de juros, inflação ou apetite ao risco. Por isso, o modelo deveria ser reavaliado periodicamente antes de ser usado em produção.

Também não foram considerados custos de transação, spread, liquidez intradiária ou slippage. Como o objetivo do projeto é melhorar alertas e não executar ordens automaticamente, essas simplificações são aceitáveis, mas devem ser reconhecidas.

7. Conclusões

O projeto demonstrou uma evolução tecnicamente coerente do B3Tracker. A solução transforma um alerta estático em uma classificação supervisionada baseada em múltiplos sinais de mercado. A arquitetura é reproduzível, modular e compatível com integração futura em Django.

A principal contribuição não é afirmar que o modelo prevê o mercado com precisão, mas sim mostrar como um sistema real pode incorporar aprendizado de máquina de forma controlada: com engenharia de features, validação temporal, análise de matriz de confusão e limiar conservador para reduzir falsos positivos.

Como próximos passos, recomenda-se testar mais ativos, comparar diferentes horizontes de previsão, avaliar custos de transação e implementar monitoramento de degradação do modelo ao longo do tempo.